

A Separable ξ -Precontraction for Interpolation-Supplemented Lattice Boltzmann Streaming on a Streamwise-Uniform Curvilinear Grid

Abstract

We describe the single most profitable algebraic simplification in the GPU implementation of an interpolation-supplemented lattice Boltzmann method (ISLBM) on a curvilinear periodic-hill grid: the streamwise (ξ) factor of the tensor-product Lagrange streaming interpolant has spatially *constant* coefficients, and can therefore be hoisted out of the per-node kernel and applied once per lattice site as a separate *precontraction pass*. For the fully three-dimensional populations this reduces the per-node interpolation cost from $7^3 = 343$ global loads and ≈ 399 fused multiply-adds to $49 + 7$ loads and ≈ 63 multiply-adds, a $\approx 6\times$ reduction on the memory-bandwidth-bound streaming kernel. We give the exact formulation, map every equation to the production CUDA code, and state precisely why the transformation is legitimate, both algebraically and in floating point.

1 Setting and notation

The solver evolves a D3Q19 distribution f_q , $q = 0, \dots, 18$, on a structured curvilinear grid whose three computational coordinates have deliberately different metric character:

coordinate	physical direction	grid map	consequence for weights
η (index i)	spanwise x	uniform, periodic	constant per $\text{sgn}(e_x)$
ξ (index j)	streamwise y	straight, uniform	constant per $\text{sgn}(e_y)$
ζ (index k)	wall-normal z	terrain-following tanh	varies with (j, k)

The streamwise map is affine, $y(j) = \Delta y (j-3)$ with $\Delta y = L_y/(N_y-1)$; the wall-normal map follows the hill profile through a tanh stretching and is inverted analytically (atanh) during preprocessing.

Streaming is semi-Lagrangian: the post-collision distribution at the *departure point* of velocity $\mathbf{e}_q$ is reconstructed by a seventh-order (7-point) Lagrange interpolation, applied as a tensor product over the three computational coordinates.

2 Mathematical formulation

2.1 The streaming interpolant

For a fully three-dimensional population ($e_x \neq 0$, $e_y \neq 0$; on this lattice $q \in \{7, \dots, 10\}$ over hill rows) the streamed value at node (i, j, k) is the triple contraction

$$f_q^{\text{str}}(i, j, k) = \sum_{a=0}^6 \sum_{b=0}^6 \sum_{c=0}^6 w_a^\eta[\text{sgn}(e_x)] w_b^\xi[\text{sgn}(e_y)] w_c^\zeta(C_q; j, k) f_q^{\text{post}}(i-3+a, j-3+b, k_0(C_q; j, k)+c), \quad (1)$$

where C_q is the (e_y, e_z) projection class of q , and $k_0(C_q; j, k)$ is the (precomputed) base index of the contiguous seven-point ζ stencil. The three weight families are *asymmetric in their spatial dependence*: w^η and w^ξ are global constants, while w^ζ is a per- (C_q, j, k) folded table.

2.2 Spatial constancy of the ξ weights

Because the streamwise map is affine, the departure offset in index space is the *same at every node*:

$$r = -\frac{e_y \delta t}{\Delta y} \quad (\text{a single global constant per } \text{sgn}(e_y)), \quad (2)$$

so the interpolation abscissa on the local stencil $\{0, \dots, 6\}$ (centred at node 3) is $t = 3 + r$, and the Lagrange coefficients

$$w_b^\xi = \prod_{\substack{m=0 \\ m \neq b}}^6 \frac{t - m}{b - m}, \quad b = 0, \dots, 6, \quad (3)$$

are *independent of* (i, j, k) . Only three rows exist: $e_y = +1$, $e_y = -1$, and $e_y = 0$. A CFL-type gate $|r| < 1$ guarantees the departure point stays inside the centred stencil, so one constant table suffices for the whole domain.

For $e_y = 0$ the offset vanishes and the weights collapse to an *exact* discrete delta,

$$r = 0 \implies w_b^\xi = \delta_{b,3} = [0, 0, 0, 1, 0, 0, 0] \implies \mathcal{L}^\xi = I, \quad (4)$$

which is why only the ten populations with $e_y \neq 0$, $q \in \{3, 4, 7, 8, 9, 10, 15, 16, 17, 18\}$, require any ξ interpolation at all.

2.3 The precontraction pass

Equation (1) is a tensor-product operator and therefore factorises,

$$\mathcal{L}_q^{3D} = \mathcal{L}^\eta \otimes \mathcal{L}^\xi \otimes \mathcal{L}^\zeta = \underbrace{(\mathcal{L}^\eta \otimes I \otimes \mathcal{L}^\zeta)}_{\text{main kernel}} \underbrace{(I \otimes \mathcal{L}^\xi \otimes I)}_{\text{precontraction pass}}. \quad (5)$$

The middle factor is applied *once per lattice site* in a separate kernel, producing the auxiliary field

$$X_q(i, j, k) = \sum_{b=0}^6 w_b^\xi [\text{sgn}(e_y)] f_q^{\text{post}}(i, j-3+b, k), \quad q : e_y \neq 0, \quad (6)$$

after which the streaming kernel evaluates only the remaining double contraction

$$f_q^{\text{str}}(i, j, k) = \sum_{a=0}^6 w_a^\eta \sum_{c=0}^6 w_c^\zeta(j, k) X_q(i-3+a, j, k_0+c). \quad (7)$$

2.4 Cost

Per node and per fully three-dimensional population:

$$\underbrace{343 \text{ loads, } \approx 399 \text{ FMA}}_{\text{fused evaluation of (1)}} \longrightarrow \underbrace{7 + 49 \text{ loads, } \approx 63 \text{ FMA}}_{\text{(6) (amortised) + (7)}} \quad (\approx 6\times). \quad (8)$$

For the $e_x = 0$, $e_y \neq 0$ populations the reduction is $49 \rightarrow 7 + 7$. Since the kernel is memory-bandwidth bound (arithmetic intensity ≈ 0.25 FLOP/byte, far below the V100 ridge point), the load reduction translates almost directly into wall-clock time.

3 Implementation: equation-to-code map

All listings are lightly trimmed excerpts of the production source at commits 30687a5 (pass introduced) and 87d0ef4 (direction-parallel consumer); file and line references are given in Table 1.

Table 1: Correspondence between equations and code.

Equation	Listing	Source location
(2),(3)	1	islbm/islbm_precompute.h:101
(4) (hard gate)	2	main.cu (startup, ISLBM block)
(6)	3	islbm/islbm_streaming.h, Islbm_PrecomputeXi_Kernel
(7)	4	islbm/islbm_streaming.h, islbm_qpar_stream_one

3.1 Constant weight table — Eqs. (2), (3)

The host builds the 3×7 table once; `d_xi` realises the constant offset $-r$ of Eq. (2) and `lagrange_7point_coeffs_host` evaluates Eq. (3) at $t = 3 - d_{xi}$.

```

1 static inline double Islbm_PrecomputeXiWeights(
2     double wxi[3][ISLBM_XI_ORDER], double dt_val)
3 {
4     const double inv_dy = (double)(NY6 - 7) / (double)LY; // = 1/dy
5     const double d_xi_mag = dt_val * inv_dy; // |r| for e_y = +-1
6     for (int idx = 0; idx < 3; idx++) {
7         const double ey = (idx == 0) ? 1.0 : (idx == 1 ? -1.0 : 0.0);
8         const double d_xi = dt_val * ey * inv_dy; // Eq. (2): global constant
9         double t = 3.0 - d_xi; // abscissa on stencil {0..6}
10        if (t < 0.0) t = 0.0;
11        if (t > 6.0) t = 6.0;
12        lagrange_7point_coeffs_host(t, wxi[idx]); // Eq. (3)
13    }
14    return d_xi_mag; // caller enforces |r| < 1
15 }

```

Listing 1: Host construction of the constant ξ table (`islbm/islbm_precompute.h`). Row 0: $e_y=+1$; row 1: $e_y=-1$; row 2: $e_y=0$ (identity).

The table is uploaded to constant memory, `__constant__ double ISLBM_L_xi_shared[3][7]`, so every thread reads it via broadcast at zero bandwidth cost.

3.2 Startup hard gates — Eq. (4) and the CFL premise

Both structural premises of the simplification are verified at startup and abort the run on violation; the reduced streaming paths are *never* allowed to execute on a table that does not satisfy them.

```

1 double islbm_wxi_h[3][ISLBM_XI_ORDER];
2 double d_xi_mag = Islbm_PrecomputeXiWeights(islbm_wxi_h, dt_global);
3 if (d_xi_mag >= 1.0) { // CFL gate: |r| < 1
4     fprintf(stderr, "[ISLBM] FATAL: |r| = dt/dy = %.6f >= 1 - xi departure\n",
5         "leaves the centered 7-point stencil;...\n", d_xi_mag);
6     MPI_Abort(MPI_COMM_WORLD, 93);
7 }
8 // Row 2 (e_y = 0) must be the value-exact identity [0,0,0,1,0,0,0]:
9 // the reduced paths (q=5,6 pure zeta; q=11..14 eta x zeta) assume it.

```

```

10 for (int s = 0; s < ISLBM_XI_ORDER; s++) {
11     const double expect = (s == 3) ? 1.0 : 0.0;
12     if (islbm_wxi_h[2][s] != expect) {          // Eq. (4), bitwise
13         fprintf(stderr, "[ISLBM] FATAL: xi_row2[%d] = %.17e != %.1f\n",
14             "identity_row_broken;...\n", s, islbm_wxi_h[2][s], expect);
15         MPI_Abort(MPI_COMM_WORLD, 95);
16     }
17 }
18 CHECK_CUDA( cudaMemcpyToSymbol(ISLBM_L_xi_shared, islbm_wxi_h,
19                                 sizeof(islbm_wxi_h)) );

```

Listing 2: Startup validation in `main.cu`: the $|r| < 1$ premise of the single constant table, and the exact identity row of Eq. (4).

3.3 The precontraction kernel — Eq. (6)

One thread per output value $X_q(i, j, k)$; the ten $e_y \neq 0$ directions are enumerated by `blockIdx.z`. Four adjacent j rows share a thread block so that their overlapping seven-row source windows are served by L1.

```

1 __global__ void Islbm_PrecomputeXi_Kernel(
2     const double * __restrict__ f_post_read,
3     double * __restrict__ xi_tmp)
4 {
5     const int i = 3 + blockIdx.x * blockDim.x + threadIdx.x;
6     const int j = 3 + blockIdx.y * blockDim.y + threadIdx.y;
7     const int nk = NZ6 - 6;
8     const int dir = blockIdx.z / nk;          // 0..9: e_y != 0 directions
9     const int k = 3 + blockIdx.z - dir * nk;
10    if (i >= NX6 - 4 || j >= d_NJD - 3 || dir >= ISLBM_XI_DIR_COUNT) return;
11
12    const int q = GILBM_MPI_XI_DIRS[dir]; // {3,4,7,8,9,10,15,16,17,18}
13    const double ey = GILBM_e[q][1];
14    const int ey_idx = (ey > 0.0) ? 0 : 1;    // constant-table row
15    const int nface = NX6 * NZ6;
16    const int index = j * nface + k * NX6 + i;
17    const int j0 = j - 3;
18    const size_t q_off = (size_t)q * (size_t)GRID_SIZE;
19
20    double out = 0.0;
21    #pragma unroll
22    for (int sj = 0; sj < ISLBM_XI_ORDER; sj++) // Eq. (6)
23        out += ISLBM_L_xi_shared[ey_idx][sj] *
24            f_post_read[q_off + (size_t)(j0 + sj) * nface + k * NX6 + i];
25    xi_tmp[(size_t)dir * (size_t)GRID_SIZE + index] = out;
26 }

```

Listing 3: `Islbm_PrecomputeXi_Kernel` (`islbm/islbm_streaming.h`): the separable ξ contraction of Eq. (6).

3.4 The consumer — Eq. (7)

Inside the streaming kernel the ten $e_y \neq 0$ populations never touch f^{post} again; they contract the stored field X_q (`xi_tmp`) over ζ , and—for the η -dependent populations $q \in \{7, \dots, 10\}$ —combine the resulting columns over η by warp-cooperative register shuffles.

```

1 const size_t xi_off = (size_t)xi_dir * (size_t)GRID_SIZE;
2
3 if (ex == 0.0) {                                     // q = 3,4,15..18: zeta pass only
4     const ISLBM_ZetaFolded c = islbm_zeta_d[ISLBM_ZetaIndex(cls, j, k)];
5     double out = 0.0;
6     #pragma unroll
7     for (int sk = 0; sk < 7; sk++)                  // inner sum of Eq. (7)
8         out += c.ws[sk] *
9             xi_tmp[xi_off + j * nface + (c.k0 + sk) * NX6 + i];
10    return out;
11 }
12 /* q = 7..10: build one zeta-contracted column per warp lane ... */
13 const ISLBM_ZetaFolded c = islbm_zeta_d[ISLBM_ZetaIndex(cls, j, k)];
14 double colA = 0.0;
15 #pragma unroll
16 for (int sk = 0; sk < 7; sk++)
17     colA += c.ws[sk] *
18         xi_tmp[xi_off + j * nface + (c.k0 + sk) * NX6 + giA];
19 /* ... colB analogously for the right halo lanes ... */
20 return islbm_warp_eta_combine(colA, colB, eta_sign, lane);
21                                     // outer eta sum of Eq. (7)

```

Listing 4: Consumer excerpt from `islbm_qpar_stream_one` (`islbm/islbm_streaming.h`, commit 87d0ef4): the residual ζ (and η) contraction of Eq. (7) applied to the precontracted field.

4 Why the simplification is legitimate

4.1 Algebraic exactness

Equation (5) is an identity of finite sums: the triple contraction (1) and the staged evaluation (6)–(7) compute the same multilinear form. No approximation is introduced; the interpolation theory (seventh-order tensor-product Lagrange) is untouched.

4.2 Legality of the hoisting: spatial invariance

Hoisting the middle factor of (5) out of the per-node kernel is only valid because w^ξ carries *no* dependence on (i, j, k) — a direct consequence of the affine streamwise map, Eq. (2). The contrast with the wall-normal direction is instructive: w^ζ depends on (j, k) through the tanh terrain-following map, so $\mathcal{L}^\zeta$ *cannot* be hoisted into a shared constant table; it remains a per-node folded-table contraction. The transformation is thus exactly the classical *loop-invariant code motion*, lifted from scalar expressions to a tensor-product operator factor, and its applicability is dictated by the grid metric, not by numerical convenience. Both structural premises ($|r| < 1$ for a single centred table; the exact identity row for $e_y = 0$) are enforced by fail-closed startup gates (Listing 2).

4.3 Profitability: redundancy elimination on a bandwidth-bound kernel

In the fused evaluation of (1), the inner ξ sum belonging to a given source column is implicitly recomputed by *every* node whose $\eta \times \zeta$ stencil window covers that column — up to $7 \times 7 = 49$ consumers per value. The precontraction computes each $X_q(i, j, k)$ exactly once and stores it (10 direction slabs of one grid field), converting a 49-fold redundancy into a single evaluation, Eq. (8).

Because the streaming kernel operates far below the machine-balance point (≈ 0.25 FLOP/byte versus ≈ 8.7 FLOP/byte on V100), wall time follows bytes moved; the measured iteration time of the streaming stage dropped accordingly, and the auxiliary storage (10 slabs) is the price paid for removing $\approx 6\times$ of the dominant memory traffic.

4.4 Floating-point effect and empirical verification

The staged evaluation re-associates the sum: the fused kernel contracted ζ innermost, whereas (6)–(7) contract ξ first. Re-association of a finite sum is not bitwise neutral in IEEE-754 arithmetic; the perturbation is bounded by the classical backward-error estimate $|\Delta| \leq \gamma_n \sum_b |w_b| |f_b|$ with $\gamma_n = nu/(1 - nu)$, i.e. a few ulps for $n \leq 343$ in double precision. This bound was confirmed empirically by a ten-step replay from an identical checkpoint with the pass switched on and off:

$$\max |\Delta f| = 8.88 \times 10^{-16}, \quad \max |\Delta \rho| = 2.44 \times 10^{-15}, \quad (9)$$

i.e. machine-epsilon-level agreement. Regression comparisons against the pre-factorisation implementation therefore use a 10^{-13} -level acceptance threshold rather than bitwise equality, while the pass itself remains switchable (`ISLBM_XI_PREPASS=0` restores the fused contraction) so that the two evaluations remain mutually verifiable.